

Roteiro Aula - 24/08

Redes Docker

Por padrão, cada contêiner possui seu próprio ambiente de rede.

Isso significa que, dentro do contêiner da aplicação Java:

localhost

representa o próprio contêiner da aplicação, e não:

- o computador do usuário;
- o contêiner do PostgreSQL;
- qualquer outro contêiner.

Para que a aplicação Java consiga encontrar o PostgreSQL, colocaremos os dois contêineres na mesma rede Docker.

Uma rede Docker do tipo **bridge** funciona como uma rede privada virtual. Contêineres conectados a ela podem se comunicar usando seus nomes como hostnames. O Docker mantém um DNS interno para fazer essa resolução de nomes. [Docker Docs — Bridge network driver](#)

Assim, a aplicação poderá acessar:

postgres-aula:5432

Parte 1 - Executar a aplicação com o Postgres

1. Executar o container do postgres

```
docker run -d --name postgres-aula -e POSTGRES_DB=auladb -e POSTGRES_USER=usuario  
-e POSTGRES_PASSWORD=senha -p 5432:5432 postgres
```

2. Configurar a aplicação para rodar com o Postgres (pom.xml)

```
<dependency>  
  <groupId>org.postgresql</groupId>  
  <artifactId>postgresql</artifactId>  
  <scope>runtime</scope>  
</dependency>
```

3. Configurar a aplicação para rodar com o postgres (application.properties)

```
spring.datasource.url=jdbc:postgresql://localhost:5432/auladb
spring.datasource.username=usuario
spring.datasource.password=senha
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

4. Executar a aplicação sem o docker, tudo funciona.
5. Executar a aplicação com o docker, não funciona.

Parte 2 - Configurar a aplicação para executar com uma rede Docker

1. Criar a rede Docker

```
docker network create -d bridge rede
```

2. Mudar o comando do postgres para executar com a rede agora

```
docker run -d --name postgres-aula -e POSTGRES_DB=auladb -e POSTGRES_USER=usuario
-e POSTGRES_PASSWORD=senha --network rede -p 5432:5432 postgres
```

3. Mudar a configuração da aplicação no application.properties

```
spring.datasource.url=jdbc:postgresql://postgres-aula:5432/auladb
spring.datasource.username=usuario
spring.datasource.password=senha
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

4. Executar a aplicação com docker, conectando na rede.

Parte 3 - Usar variáveis de ambiente

1. Para não ficar tendo que trocar os valores toda hora, podemos usar variáveis de ambiente no arquivo application.properties

```
spring.datasource.url=jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:auladb}
```

```
spring.datasource.username=${DB_USER:usuario}
```

```
spring.datasource.password=${DB_PASSWORD}
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

Parte 4 - Exercício - Subir tudo isso na máquina da AWS